

情報科学演習 D

課題 2

担当教員 : 梶井 晃基
提出者 : 鶴見 俊介
所属/学年 : 基礎工学部 情報科学科 3 年 計算機科学コース
学籍番号 : 09B23057
電子メール : u299151f@ecs.osaka-u.ac.jp

提出日 : 2025 年 11 月 14 日

締切日 : 2025 年 11 月 14 日

1 システムの仕様

Pascal 風言語で記述されたプログラムのトークン列ファイル (ts ファイル) を第一引数として入力し、プログラムの構文が正しいかどうか判定する構文解析プログラムを作成する。構文が正しい場合は文字列 "OK" を返し、構文が正しくない場合は文字列 "Syntax error: line" という文字列とともに、最初の誤りを見つけた行番号を返す。入力ファイル内に複数の誤りが含まれる場合は、最初に見つけた誤りの行番号のみ返す。また、入力ファイルが見つからない場合は、文字列 "File not found" を返す。

なお、本構文解析器は構文が正しいと判定できた場合に限り、後続の意味解析に用いるための抽象構文木 (AST) を内部に構築する (根は `Program`, 主な子は `Block` と `Compound` 等). 各 AST ノードは少なくとも種別と先頭・末尾行の位置情報を保持する. エラー検出時は解析を中止するため, AST は確定しない.

2 課題達成の方針と設計

2.1 中核アイデアと設計原則

本実装は、字句解析済みトークン列を入力とし、トップダウンの再帰下降型で構文木 (AST) を生成する。文・宣言・式などの各非終端記号の解析は、それぞれ `parseStmt()`, `parseDecl()`, `parseExpr()` のような関数に対応付け、規則=関数の直観的な呼び出し構造を採る。

構文解析は、左再帰性を含まない無曖昧な文法で、1トークンの先読み(LL(1))を用いる。指導書のEBNFは、文と基本文の分岐先決定に2トークンの先読み(LL(2))が必要なため、LL(1)で解析可能な文法に修正する必要がある。実装は`peek()`による定数回の確認で解決し、バックトラックは行わないことで決定性と線形時間性を維持する。

また、構文解析を進めながら、トークン列を木構造に保存してASTを作成していく。ASTの作成は、構文解析においては必須ではないが、後の意味解析以降の処理で再利用可能な形となることに加え、軽量で後続の処理と疎結合となり拡張性をもつメリットがあると考えられる。

以上により，文法の明確化（左再帰除去と先読み設計）と実装の単純性（関数と規則の対応，決定的分岐），および将来拡張性（AST 中心設計）を同時に満たす方針とした．

2.2 データ構造

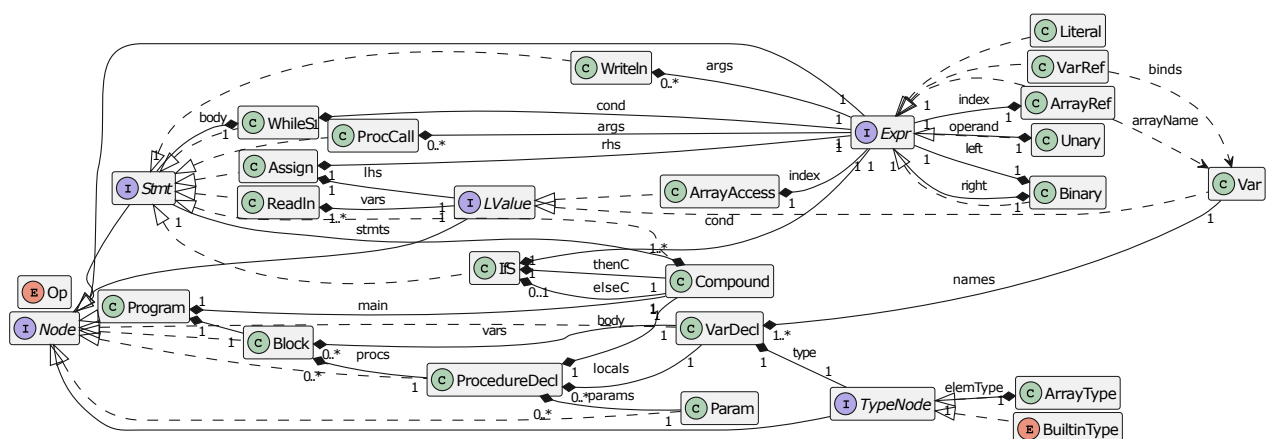


図 1: AST のクラス図

まず、TsSource が入力ファイル内文字列の各行を、名前・トークン種・トークン ID・行番号の4つの要素に分解して、TsToken 型の List に順に格納していくことで、トークンに関する情報は保存される。このようにして保存された List を順に参照していくことで構文解析を行い、AST を作成していく。

本課題の AST は、Node を共通基底とする木構造であり、各ノードは種別、子ノード、行番号をもつ。種別は、構造系 (Program・Block・Compound)、宣言系 (VarDecl・ProcedureDecl・Param・ArrayType・BuiltinType)、文系 (Assign・IfS・WhileS・ProcCall・Readln・Writeln)、式系 (Binary・Unary・Literal・VarRef・ArrayRef) に大別する。左辺値は LValue として抽象化し、Var と ArrayAccess が実装する。演算子種別は Op 列挙で保持し、優先順位・結合性は木構造 (例: Binary(op=ADD, left=..., right=...)) に反映する。これらの木構造のクラス関係をまとめたものが図 1 である。

2.3 解析アルゴリズム

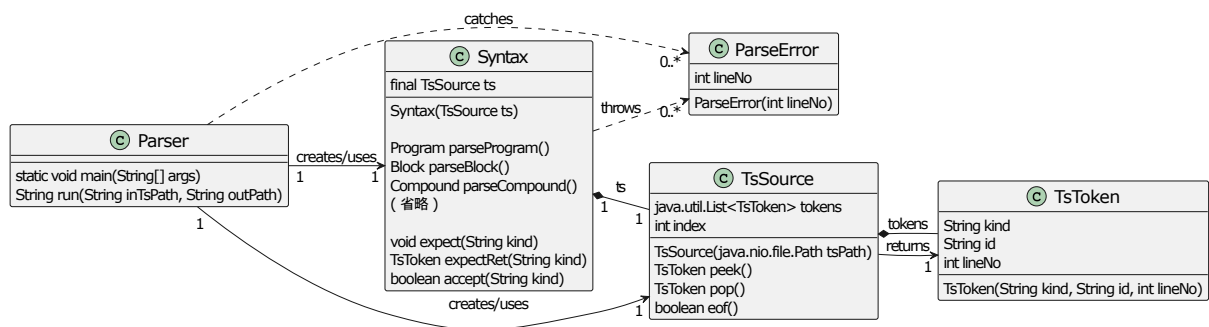


図 2: 構文解析のクラス図

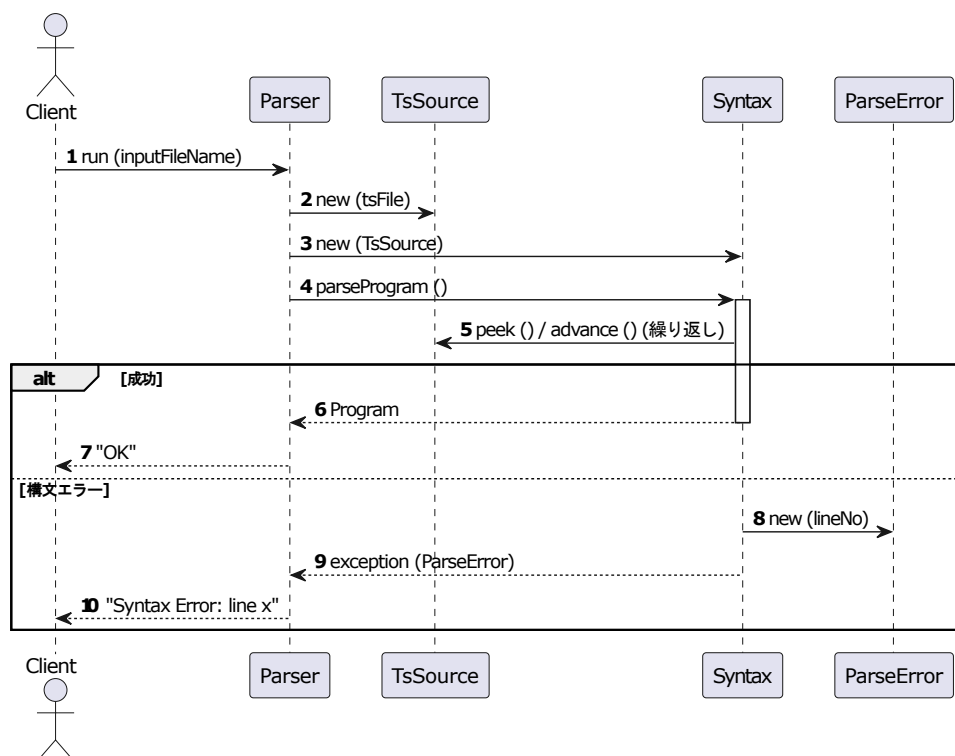


図 3: 構文解析のシーケンス図

図2は全体の処理におけるクラス間の関係を表したクラス図であり、図3はそれらのクラスが処理を行う流れを表したシーケンス図である。これらによる呼び出し関係と処理の流れをまとめると次ようになる。なお、以下の処理番号は図3中の番号と対応している。

1. Client が `Parser.run(inputFileName)` を起動する。
2. Parser は入力 TS ファイルから `TsSource` を生成する。
3. Parser は `TsSource` を引数に `Syntax` を生成する。
4. Parser は `Syntax.parseProgram()` を呼び出し、解析を開始する。
5. Syntax は `TsSource.peek()/advance()` を繰り返し用いてトークンを消費し、対応する AST を構築する。解析が成功したら6へ、失敗したら8へ進む。
6. Syntax は `Program` ノードを Parser に返す。
7. Parser は "OK" を Client に返して終了する。
8. Syntax は `ParseError(lineNo)` を生成する。
9. Syntax は Parser に例外 `ParseError(lineNo)` を送出する。
10. Parser は "Syntax Error: line x" を Client に返して終了する。

3 実装プログラム

全体の処理は図3の流れによって行われるが、この内の Parser が `Syntax.parseProgram()` を呼び出すことで構文解析を開始する。以下のプログラム1は、`parseProgram()` の実装内容である。`parseProgram()` は、「プログラム = "program" プログラム名 ";" ブロック 複合文 "."」という順番で記述されているかどうかを検査する。以降は「非終端＝関数」の対応で、上位規則から下位規則へ決定的に遷移する。ここで呼び出されている関数 `expect(String)` は引数で渡された文字列がトークン名と一致しているかどうかを判定し、一致しているならばトークンを消費、一致していないならば例外 `ParseError` を送出する。`expectRet(String)` は引数で渡されたトークン種と一致しているかどうかを判定し、一致しているならばそのトークンを返す `TsToken` 型の関数であり、一致しないならば例外 `ParseError` を送出する。

また、プログラムはブロックと複合文を構成要素としているが、これらも同様に文法的に正しく記述されているか確認する必要がある。`parseBlock()` は宣言列、`parseCompound()` は `Stmt` の1個以上の並びを担当する。これらの関数も `parseProgram` と同様に、各非終端の検査に対応した関数を呼び出す。各関数は期待されるトークン列を `expect` などで検証・消費しながら対応する AST ノードを生成し、復帰時に親子関係を確定する。

以上を各規則に対して再帰的に適用することで、図3の流れに沿って入力全体を一意に解析し、最終的に `Program` を根とする AST を得る。

プログラム 1: `parseProgram`

```
1 public Program parseProgram() {
2     expect("SPROGRAM");
3     var nameTok = expectRet("SIDENTIFIER");
4     expect("SSEMICOLON");
5     Block block = parseBlock();
6     Compound main = parseCompound();
7     expect("SDOT");
8     return new Program(nameTok.name, block, main, nameTok.lineNo);
9 }
```

4 考察や工夫点

4.1 EBNF による生成規則

指導書の EBNF を修正して、LL(1) で解析が可能になるような文法を作成した。これによって、LL(2) によるバックトラックが生じることなく構文解析を完了することができる。

修正した部分は、基本文と文の生成規則である。基本文は、「基本文 = 代入文 | 手続き呼出し文 | 入出力文 | 複合文」として定義されているが、代入文と手続き呼び出し文はともに先頭が識別子であるため、先頭トークンが識別子だった場合に分岐先が確定しない。したがって、以下のように非終端「代入または手続き」を追加して 1 トークンの先読みで解析ができるようにする。

基本文 = 識別子 代入または手続き | 入出力文 | 複合文
代入または手続き = ":"=" 式 | ["(" 式の並び ")"]

同様に、文の生成規則における if 文への分岐も、else の有無が 1 トークンの先読みでは確定しない文法になっているため、次のように非終端「else 文」を追加して修正する。

文 = 基本文 | "if" 式 "then" 複合文 else 文 | "while" 式 "do" 複合文
else 文 = ["else" 複合文]

また、左再帰性を含む生成規則も LL(1) での解析が不能である。指導書の EBNF には左再帰性は含まれていないが、「 $E \rightarrow E+T \mid T$ 」のようにして含まれていた場合は、「 $E \rightarrow TE'$ 」「 $E' \rightarrow \varepsilon \mid +TE'$ 」のようにして左再帰性を除去する必要がある。なぜならば、「 $E \rightarrow E+T \mid T$ 」のように左再帰性があると、 $\text{First}(E)=\text{First}(T)$ となり、E の分岐先が 1 トークンの先読みでは確定せず、 $E \rightarrow E+T$ の生成を無限に選択してしまうためである。このように、LL(1) による簡潔な解析を実装するためには、左因子分解と左再帰性除去による無曖昧な文法が必要であるとわかる。

4.2 AST の役割・将来性

AST はプログラムの構文構造を失わずに最小情報で表す中間表現である。文法規則を木に写像することで、整形出力・可視化・回帰テストの基盤になり、将来の意味解析やコード生成を疎結合に追加できる。一方で属性を増やしすぎると重くなるため、今回の最小設計（種別・子列・位置）に抑えた判断は妥当だと考える。構文解析の段階では AST の利点は限定的だが、Visitor パターン等により後続機能（ダンプ整形、静的検査、IR 生成）の拡張余地が確保できた点は大きい。

5 感想

今回の実装では、特に AST の設計と生成で苦労した。文法を素直に木に写像するだけでもノード粒度・親子関係・位置情報の扱いに迷いが生じ、最小属性に絞る判断が難しかった。一方で、AST をオブジェクトとして保持できるようにしたことで、整形出力・可視化・テスト、再解析や将来の意味解析・コード生成の土台として再利用しやすくなった点は大きな収穫である。全体を通じて、言語処理工学の授業で学んだ再帰下降、左因子分解、先読み設計といった基礎知識が直接役立ち、実装の見通しとデバッグ効率の向上に繋がった。